

Installation et de Configuration : Serveur VPN WireGuard - Nathaniel T

1. Contexte et Architecture Réseau

L'objectif de cette mise en place est de permettre à des postes distants (depuis le WAN) d'accéder de manière sécurisée au réseau local de l'infrastructure (zone MZ, INFO, Guest) via un tunnel VPN WireGuard basé sur un serveur Debian.

Topologie IP

- **Serveur VPN (Debian) :** 192.168.150.3
- **Passerelle Serveur VPN :** 192.168.150.1 (Masque : 255.255.255.240 ou /28)
- **Réseau Local cible (MZ) :** 192.168.110.0/27 (Passerelle : 192.168.110.1)
- **Serveur DNS Local :** 192.168.110.10
- **Réseau du Tunnel VPN :** 10.8.0.0/24

2. Préparation du Serveur (Routage IPv4)

Afin que le serveur Debian puisse transférer les paquets du client VPN vers les autres réseaux locaux (fonction de routeur), le transfert IP (IP Forwarding) a été activé.

1. Création d'un fichier de configuration dédié :

```
sudo nano /etc/sysctl.d/99-wireguard.conf
```

2. Ajout du paramètre d'activation :

```
net.ipv4.ip_forward=1
```

3. Application immédiate des nouveaux paramètres système :

```
sudo systemctl --system
```

3. Installation et Génération des Clés Cryptographiques

WireGuard repose sur un échange de clés publiques/privées.

1. Installation des paquets nécessaires :

```
sudo apt update  
sudo apt install wireguard iptables
```

2. Passage en mode administrateur (Root) pour accéder au répertoire sécurisé :

```
sudo su -  
cd /etc/wireguard
```

3. Sécurisation des permissions de création de fichiers :

```
umask 077
```

4. Génération des paires de clés (Serveur et Client) :

```
wg genkey | tee server_private.key | wg pubkey > server_public.key  
wg genkey | tee client_private.key | wg pubkey > client_public.key
```

4. Configuration du Serveur VPN

L'interface virtuelle du serveur a été configurée dans le fichier

`/etc/wireguard/wg0.conf` .

```
[Interface]  
# Adresse IP du serveur dans le tunnel  
Address = 10.8.0.1/24
```

```

ListenPort = 51820
PrivateKey = <CONTENU_DE_SERVER_PRIVATE_KEY>

# Règles NAT et de routage pour l'accès aux réseaux LAN
# Note : eth0 est à remplacer par le nom réel de l'interface (ex: ens18)
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT; iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT; iptables -t nat -D POSTROUTING -o eth0 -j MASQUERADE

[Peer]
# Configuration pour le client distant
PublicKey = <CONTENU_DE_CLIENT_PUBLIC_KEY>
AllowedIPs = 10.8.0.2/32

```

5. Configuration du Client Distant

La configuration suivante est à déployer sur le poste client (Windows, macOS, Linux, etc.) souhaitant se connecter de l'extérieur.

```

[Interface]
Address = 10.8.0.2/24
PrivateKey = <CONTENU_DE_CLIENT_PRIVATE_KEY>
DNS = 192.168.110.10

[Peer]
PublicKey = <CONTENU_DE_SERVER_PUBLIC_KEY>
Endpoint = <ADRESSE_IP_PUBLIQUE_DU_ROUTEUR>:51820
# Split Tunneling : On autorise le trafic vers le tunnel, le réseau VPN et le réseau MZ
AllowedIPs = 10.8.0.0/24, 192.168.150.0/28, 192.168.110.0/27

```

6. Activation et Lancement du Service

Démarrage du tunnel sur le serveur Debian et activation au démarrage du système :

```
# Démarrage de l'interface wg0
sudo systemctl start wg-quick@wg0

# Activation automatique au redémarrage
sudo systemctl enable wg-quick@wg0

# Vérification du statut
sudo wg show
```

7. Actions requises côté Pare-feu / Routeur

Pour finaliser l'architecture, une règle de redirection de port (Port Forwarding ou DNAT) doit être configurée sur le routeur principal de l'infrastructure :

- **Protocole** : UDP
- **Port source (WAN)** : 51820
- **IP de destination (LAN)** : 192.168.150.3
- **Port de destination** : 51820